

RAG Scan Stack Simple Architecture

Contents

1	RAG Scan Stack - Architecture Overview	1
1.1	System Overview	2
1.1.1	Core Principle	2
1.2	Basic Architecture	2
1.3	Technology Stack	2
1.3.1	Core Components	2
1.3.2	Security Scanners (Each in separate container)	2
1.4	API-First Design	3
1.4.1	Everything Has an API Endpoint	3
1.4.2	Direct Scanner APIs	3
1.5	MCP Tool Integration	3
1.5.1	What is MCP?	3
1.5.2	Available MCP Tools (26 total across 6 servers)	3
1.5.3	MCP Server Endpoints	3
1.6	Data Flow	4
1.6.1	Simple Scan Workflow	4
1.6.2	Database Schema (Simplified)	4
1.7	Remote Access & Tunneling	4
1.7.1	SSH Tunnels	4
1.7.2	WireGuard VPN	4
1.8	Deployment	4
1.8.1	Single Host Deployment	4
1.8.2	Environment Requirements	4
1.9	Security Model	5
1.9.1	Container Isolation	5
1.9.2	API Security	5
1.9.3	Data Protection	5
1.10	Integration Points	5
1.10.1	Import/Export	5
1.10.2	External APIs	5
1.11	Monitoring & Health	5
1.11.1	Service Health	5
1.11.2	Logging	5
1.12	Summary	6

1 RAG Scan Stack - Architecture Overview

Simple, API-Driven Security Testing Platform

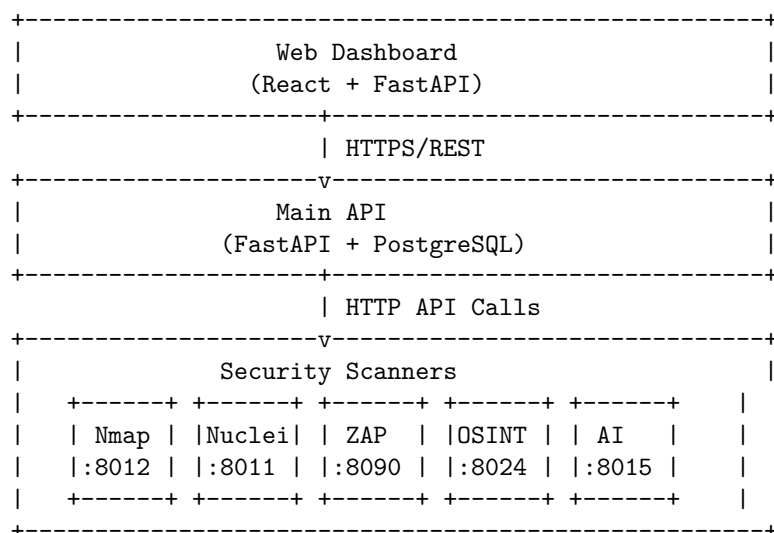
1.1 System Overview

RAG Scan Stack is a containerized security testing platform where **everything is accessible via REST APIs** and exposed through **MCP (Model Context Protocol) tools** for AI automation.

1.1.1 Core Principle

- **API-First Design:** Every function accessible via REST endpoints
 - **MCP Integration:** All capabilities exposed as AI-callable tools
 - **Container Architecture:** Each scanner runs in isolated Docker containers
 - **Unified Data Model:** All findings stored in PostgreSQL with consistent schema
-

1.2 Basic Architecture



1.3 Technology Stack

1.3.1 Core Components

- **Database:** PostgreSQL (findings, assets, configurations)
- **API Server:** FastAPI (main coordination and data management)
- **Web Interface:** React + TypeScript (dashboard for manual use)
- **Container Runtime:** Docker + Docker Compose

1.3.2 Security Scanners (Each in separate container)

- **Nmap** (Port scanning): `nmap_scanner:8012`
 - **Nuclei** (Vulnerability scanning): `nuclei-runner:8011`
 - **ZAP** (Web application scanning): `zap:8090`
 - **OSINT** (Reconnaissance): `osint-runner:8024`
 - **Playwright** (Modern web apps): `playwright-scanner:8014`
 - **Brutus** (Credential testing): `brutus-runner:8026`
-

1.4 API-First Design

1.4.1 Everything Has an API Endpoint

Scan Management:

```
POST /run_masscan_nmap      # Start port scan
POST /jobs/masscan-nmap/upload # Upload scan results
GET  /scans/{id}           # Check scan status
POST /ingest/nmap           # Process scan results
POST /ingest/nuclei         # Process vuln scan results
```

Finding Management:

```
GET  /assets                # List discovered assets
GET  /credentials           # List credential findings
DELETE /findings/bulk       # Bulk delete findings
GET  /ports/open            # List open ports
```

Export Formats:

```
GET  /export/sarif          # SARIF v2.1.0 export
GET  /export/har             # HAR format export
GET  /export/burp            # Burp Suite XML export
POST /findings/export        # Custom export formats
```

1.4.2 Direct Scanner APIs

Each scanner exposes its own REST API: - https://nmap_scanner:8012/ - Port discovery service - <https://nuclei-runner:8011/> - Vulnerability testing service - <https://zap:8090/> - Web application scanning - <https://osint-runner:8024/> - Reconnaissance tools

1.5 MCP Tool Integration

1.5.1 What is MCP?

Model Context Protocol - allows AI agents to call platform functions as “tools”

1.5.2 Available MCP Tools (26 total across 6 servers)

Scanning Tools: - `scan_ports()` - Run Nmap port scans - `scan_vulnerabilities()` - Run Nuclei templates - `scan_web_app()` - ZAP web application testing - `scan_subdomains()` - OSINT reconnaissance

Data Management: - `search_findings()` - Query vulnerability database - `get_asset_info()` - Asset and service details - `export_results()` - Generate reports

System Control: - `check_service_health()` - Monitor scanner status - `manage_scan_queue()` - Control scan execution

1.5.3 MCP Server Endpoints

```
http://localhost:9016 # pentest-sessions (scan coordination)
http://localhost:9017 # pentest-scanning (nmap, nuclei tools)
http://localhost:9018 # pentest-recon (OSINT, subdomain enum)
http://localhost:9019 # pentest-exploit (exploit execution)
http://localhost:9020 # pentest-credentials (credential testing)
http://localhost:9021 # scan-pipelines (automated workflows)
```

1.6 Data Flow

1.6.1 Simple Scan Workflow

1. **API Request** POST /run_masscan_nmap with target IPs
2. **Container Execution** Nmap scanner container runs scan
3. **Result Processing** ETL parser normalizes Nmap XML output
4. **Database Storage** Findings stored in PostgreSQL via POST /ingest/nmap
5. **API Response** Results available via GET /assets and GET /ports/open

1.6.2 Database Schema (Simplified)

```
-- Core Tables
assets          # Target IPs and hostnames
ports           # Discovered services
vulns           # Vulnerability findings
web_findings    # Web application issues
recon_findings  # OSINT discoveries

-- Metadata
scan_runs       # Scan execution records
engagements     # Project organization
```

1.7 Remote Access & Tunneling

1.7.1 SSH Tunnels

- **Purpose:** Route scans through compromised hosts
- **Implementation:** autossh containers with SOCKS5 proxies
- **API:** POST /ssh/connect with credentials

1.7.2 WireGuard VPN

- **Purpose:** Modern VPN alternative to SSH
 - **Implementation:** WireGuard server container
 - **API:** POST /wg/peers to create VPN configs
-

1.8 Deployment

1.8.1 Single Host Deployment

```
git clone <repository>
docker compose up -d
curl https://localhost:3002 # Web interface (HTTPS)
curl https://localhost:8000/health # API health check
```

1.8.2 Environment Requirements

- **Minimum:** 8GB RAM, 4 CPU cores, 100GB storage
 - **Recommended:** 16GB RAM, 8 CPU cores, 500GB SSD
 - **Network:** Internet access for scanner updates
 - **Optional:** NVIDIA GPU for AI features
-

1.9 Security Model

1.9.1 Container Isolation

- Each scanner runs in separate container
- Minimal privileges (no root where possible)
- Network segmentation via Docker networks

1.9.2 API Security

- API key authentication
- HTTPS encryption for all communication
- Rate limiting and request validation

1.9.3 Data Protection

- Database encryption at rest
 - Secure credential storage
 - No plaintext secrets in containers
-

1.10 Integration Points

1.10.1 Import/Export

- **Nessus:** Import `.nessus` files via `POST /ingest/nessus`
- **Burp Suite:** Export to XML sitemap via `GET /export/burp`
- **HAR:** HTTP Archive format export via `GET /export/har`
- **SARIF:** Security standard export via `GET /export/sarif`

1.10.2 External APIs

- **SHODAN:** Internet-wide scanning data
 - **VirusTotal:** File and URL analysis
 - **CVE Databases:** Vulnerability intelligence
-

1.11 Monitoring & Health

1.11.1 Service Health

- **Health Endpoint:** `GET /health` - All service status
- **Individual Services:** Each scanner has `/health` endpoint
- **Database:** Connection pool and query performance monitoring

1.11.2 Logging

- **Container Logs:** `docker compose logs <service>`
 - **Application Logs:** Structured JSON logging
 - **Audit Trail:** All API calls logged with user attribution
-

1.12 Summary

RAG Scan Stack is fundamentally:

1. **API-Driven:** Every function accessible via REST endpoints
2. **Container-Based:** Isolated, scalable scanner services
3. **MCP-Enabled:** AI agents can call any platform function
4. **Data-Centric:** Unified PostgreSQL storage with consistent schema
5. **Integration-Ready:** Standard formats for import/export

The architecture prioritizes simplicity and API accessibility over complexity.

This architecture enables both manual security testing via the web interface and fully automated testing via AI agents using the MCP tools.